EXPERIMENT NO 01

18-10-2026

NAME: YUGITHA B

REG NO: 192424102

SLOT: A

SUBJECT CODE: CSA4701

SUBJECT: DEEP LEARNING

SEMESTER: MS2-SEPTEMBER-2026

Vector and Matrix Operations for Deep Learning

SCENARIO

A neural network receives input features representing a student's study hours, attendance, and assignment scores. Create vectors and matrices to represent the data and perform matrix multiplication to calculate weighted outputs.

AIM

To implement vector and matrix operations using Python and NumPy for a deep learning scenario involving student study hours, attendance, and assignment scores, and to calculate weighted outputs through matrix multiplication.

ALGORITHM

1. Import the required Python libraries.
2. Create student data and represent it as vectors and a matrix.
3. Define weights and perform matrix multiplication.
4. Preprocess the data using Min-Max normalization.
5. Calculate the final weighted scores.
6. Display summary tables, plots, and score comparisons.
7. Identify the highest-scoring student and conclude the results.

SOURCECODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
# Create student dataset
df = pd.DataFrame({
    'Student': ['S1', 'S2', 'S3', 'S4', 'S5'],
    'Study_Hours': [2, 5, 3, 8, 6],
    'Attendance': [60, 80, 70, 95, 85],
    'Assignment_Score': [45, 75, 60, 92, 80]
})

# Display summary table
print("STUDENT SUMMARY TABLE")
display(df)

# Create input matrix
X = df[['Study_Hours', 'Attendance', 'Assignment_Score']].values

# Define weight vector
W = np.array([0.3, 0.3, 0.4])

# Calculate scores before preprocessing
df['Before_Preprocessing'] = X @ W

# Plot before preprocessing
df.set_index('Student')[['Study_Hours', 'Attendance',
                         'Assignment_Score']].plot(
    kind='bar', figsize=(9, 4), title='Before Preprocessing'
)
plt.ylabel('Values')
plt.tight_layout()
plt.show()
```

```
# Preprocess using Min-Max normalization
X_scaled = (X - X.min(axis=0)) / (X.max(axis=0) - X.min(axis=0))

# Calculate scores after preprocessing
df['Final_Score'] = (X_scaled @ W) * 100

# Display final summary table
print("SUMMARY TABLE AFTER PREPROCESSING")
display(df.round(2))

# Plot after preprocessing
pd.DataFrame(X_scaled, columns=[
    'Study_Hours', 'Attendance', 'Assignment_Score'
], index=df['Student']).plot(
    kind='bar', figsize=(9, 4), title='After Preprocessing'
)
plt.ylabel('Normalized Values (0-1)')
plt.tight_layout()
plt.show()
# Plot final student scores
plt.figure(figsize=(8, 4))
plt.bar(df['Student'], df['Final_Score'])
plt.title('Final Weighted Student Scores')
plt.xlabel('Student')
plt.ylabel('Score out of 100')
plt.ylim(0, 100)
plt.tight_layout()
plt.show()

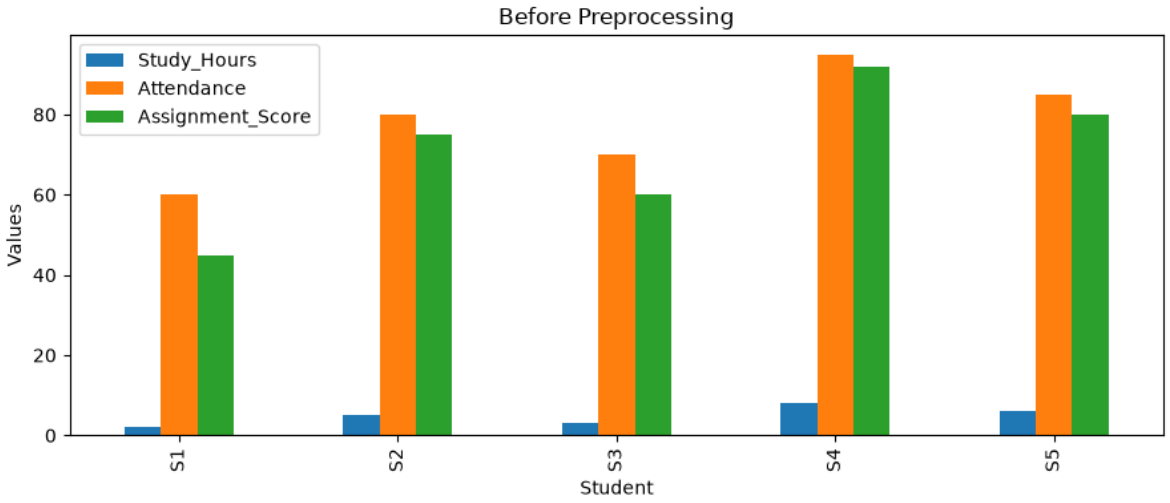
print("Final Scores:")
```

```
display(df[['Student', 'Final_Score']].round(2))
```

OUTPUT

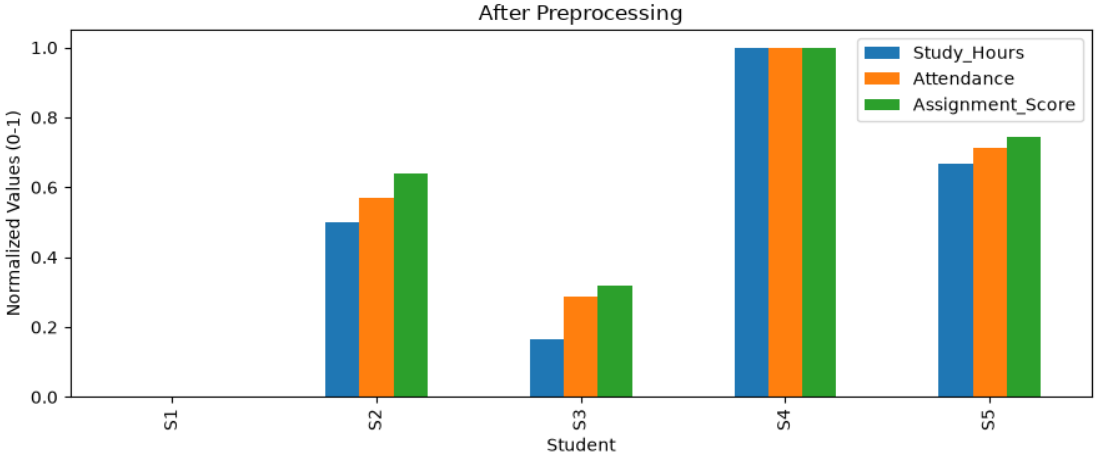
STUDENT SUMMARY TABLE

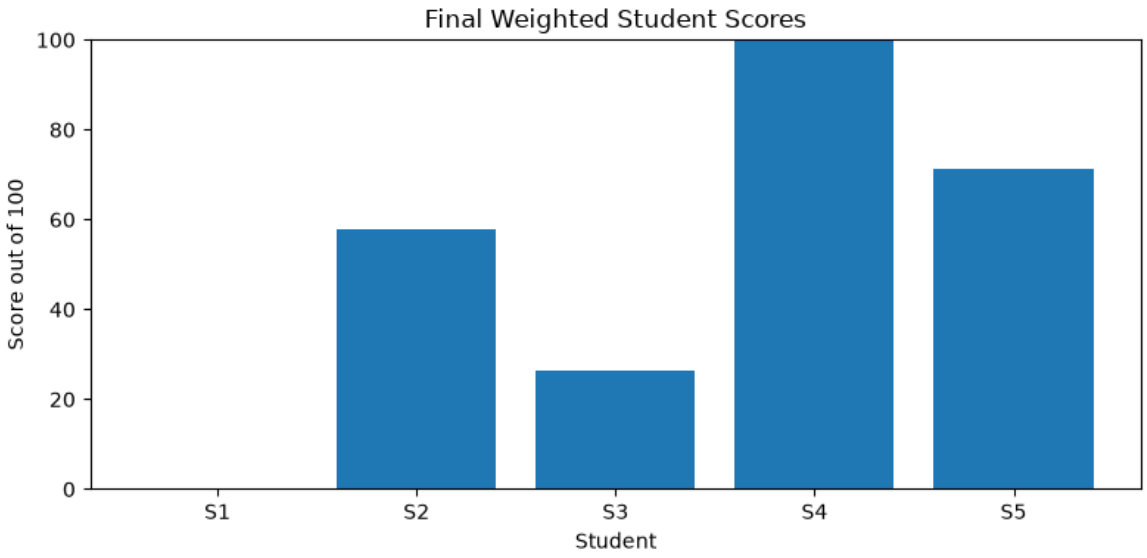
	Student	Study_Hours	Attendance	Assignment_Score
0	S1	2	60	45
1	S2	5	80	75
2	S3	3	70	60
3	S4	8	95	92
4	S5	6	85	80



SUMMARY TABLE AFTER PREPROCESSING

	Student	Study_Hours	Attendance	Assignment_Score	Before_Preprocessing	Final_Score
0	S1	2	60	45	36.6	0.00
1	S2	5	80	75	55.5	57.67
2	S3	3	70	60	45.9	26.34
3	S4	8	95	92	67.7	100.00
4	S5	6	85	80	59.3	71.22





Final Scores:

	Student	Final_Score
0	S1	0.00
1	S2	57.67
2	S3	26.34
3	S4	100.00
4	S5	71.22

RESULT

Thus, vector and matrix operations were successfully performed using Python and NumPy. The student data was preprocessed, and matrix multiplication was used to calculate weighted scores. Summary tables and plots were generated to compare the results before and after preprocessing, demonstrating the importance of matrix operations in deep learning.